Keagan Coetzee

U25122003

Task1:

1.5) using an if else chain would mean you would have to modify the factory code every time a new connector is added which violates the open closed principle. The factory method design pattern rectifies this because it defines an interface for creating an object but then lets the subclasses decide exactly which class to instantiate. By using subclasses instead of instantiation in this practical Kudu Freight can add new data by extending the system with a new Concrete creator class rather than changing the existing code.

1.6) Connector: Product which defines the product interface for the factory method to create

CsvConnector: Concrete Product which implements the interface for the product.

ConnectorFactory: Creator which declares the factory method which then returns a product

CsvFactory: Concrete Creator that overrides the factory method to return an instance of a specific product.

Task2:

2.5) A shallow copy only copies memory addresses, meaning both objects point to the same data whereas a deep copy creates a completely separate duplicate of that data. The clone() operation creates a deep copy. The Prototype pattern is designed to create new objects by copying a prototype instance. The clone must be independent so that the new object can be modified or maintained without accidentally altering the master prototypes state.

2.6) The registry returns a clone because the client requires a new object. The prototype must clone itself to do this. If the registry returned the stored pointer then many pipelines would be sharing the exact same transformation object in memory, if one pipeline modified the object's state it would break the other pipeline's processing and all pipelines would be interfering with each other's data.

Task3:

3.5) The run() method acts as the Template Method which gives the skeleton of an algorithm. It is deliberately not virtual to ensure that the overall structure of the algorithm cannot be changed which would violate the skeleton algorithm that this design pattern provides. If a subclass did override it a programmer could break the sequence violating the pattern's intent of letting the subclasses only redefine certain steps and not the order of those steps.

3.6) run(): Template Method

connect(), transform(): Concrete steps (needs to be implemented in Abstract class)

extract(), load(): Primitive operations (abstract operations that need to be defined by concrete class)

The template method defines the behavioural skeleton of the pipeline, when things happen ensuring connect, extract, transform and load happen sequentially in that order. The factory method handles the creational aspect of building the connectors (how the object is made). They do not overlap however Factory methods are often called by template methods to get objects during the execution of the skeleton.

Task4

4.4) Narrow interface: this protects against access to internal state, the caretaker only sees the narrow interface therefore it can safely hold and pass the memento to other objects never examining its contents.

Wide interface: This provides full access to all the data in order to restore the objects previous state. The Originator sees the wide interface allowing full access to the mementos internal state.

4.5) Before a crash occurs the pipeline calls createCheckpoint() to capture a snapshot of the current internal state. This snapshot/checkpoint is passed to the manager and uses save() to hold it. If the system crashed during the load() phases and restarts the manager uses undo() to retrieve the saved checkpoint. The pipeline uses restore() to pass the checkpoint in t revert it back to is state before the crash occurred.

4.6) Pipeline: the Originator (creates a memento containing a snapshot of the state)

RunCheckpoint: Memento (which stores the internal state of the Originator object.

CheckpointManager: Caretaker (responsible for safekeeping previous state)